

CS 550 Operating Systems

Spring 2026

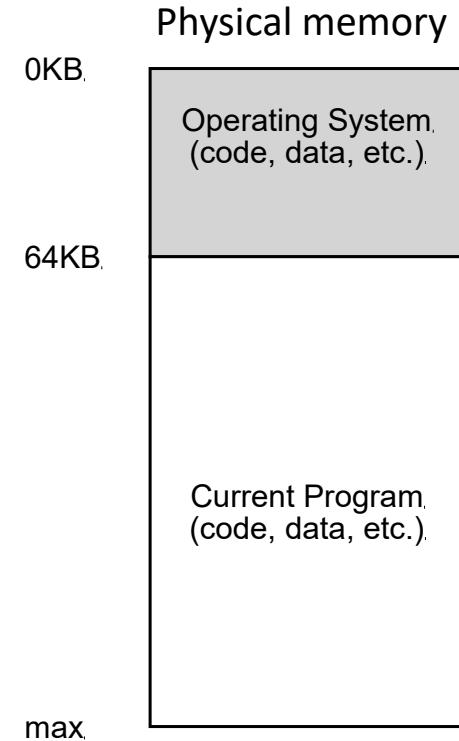
Address spaces and memory API

Main Memory

- Main memory is conceptually very simple
 - Code sits in memory
 - Data is either on a stack or a heap
 - Everything gets accessed via pointers (with C)
 - Data can be written to or read from long term storage
- Memory is a simple and obvious device
 - So why is memory management one of the most complex features in modern OSes?

Early systems: lack of memory abstraction

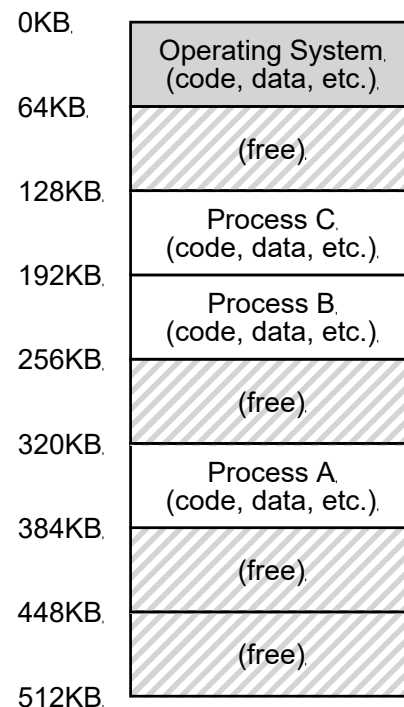
- Earlier systems did not provide much of an abstraction of (physical) memory
 - OS == library that sat in memory starting from physical address 0
 - Just one running program that sat in memory starting from physical address 64K



Multiprogramming and time-sharing systems

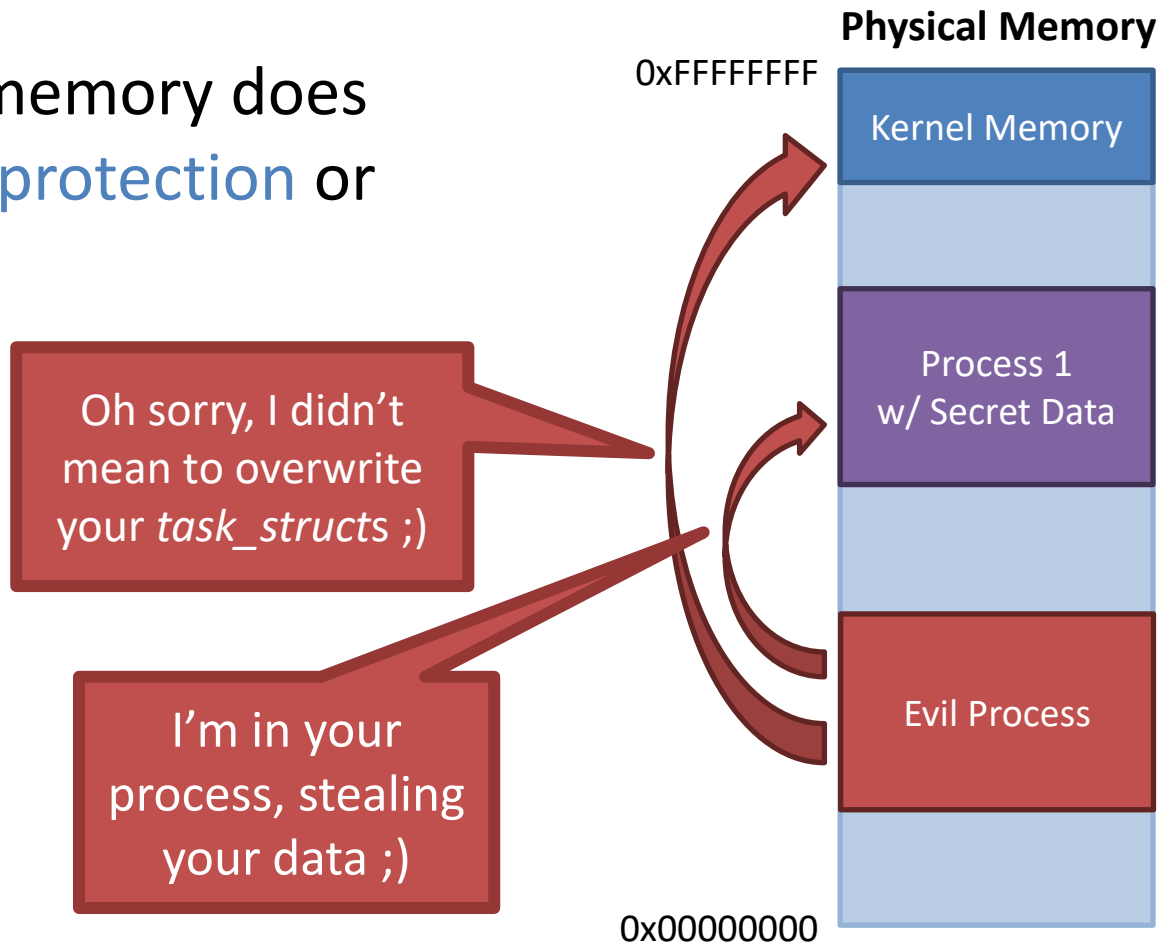
- Multiple concurrent processes
 - increase utilization
 - time sharing & interactive
- One way to implement the crude sharing of the machine: run one process (giving it full access to all memory) --> stop it, save all of its state to disk --> load another process
 - Too slow
- Leave processes in memory and switch between them (right figure)

What are the problems of this early design?



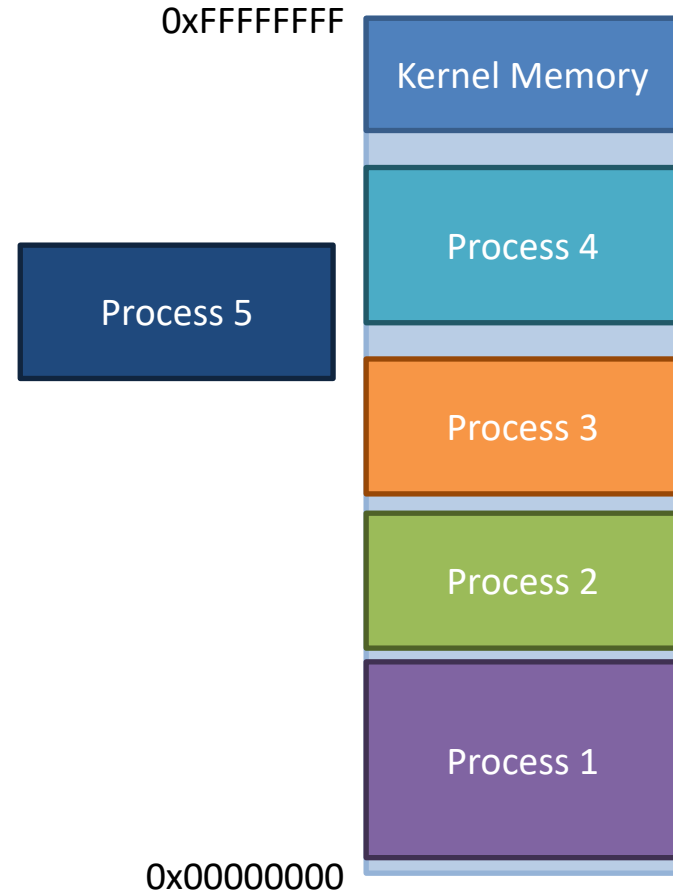
Problem 1: lack of protection and isolation

- Physical memory does not offer **protection** or **isolation**



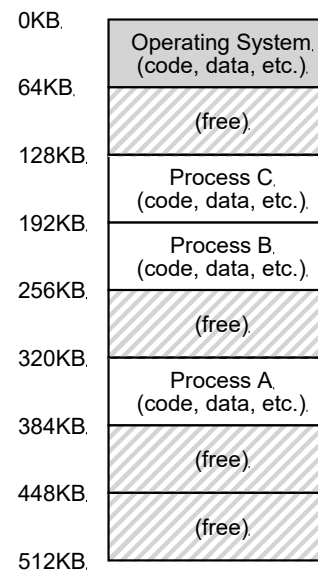
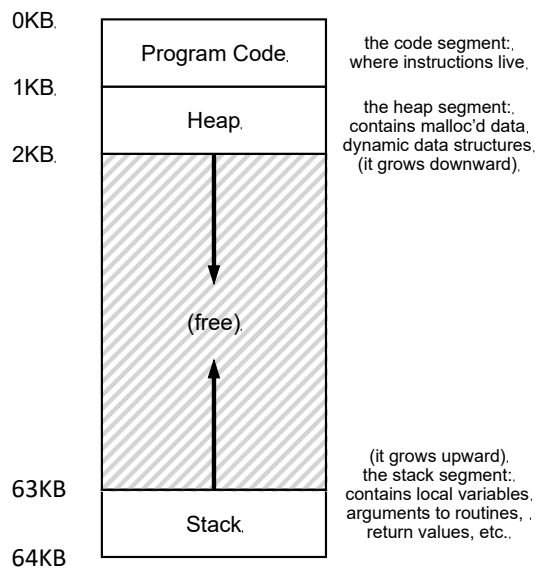
Problem 2: physical memory has limited size

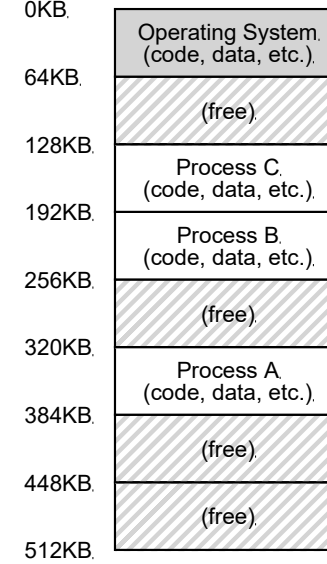
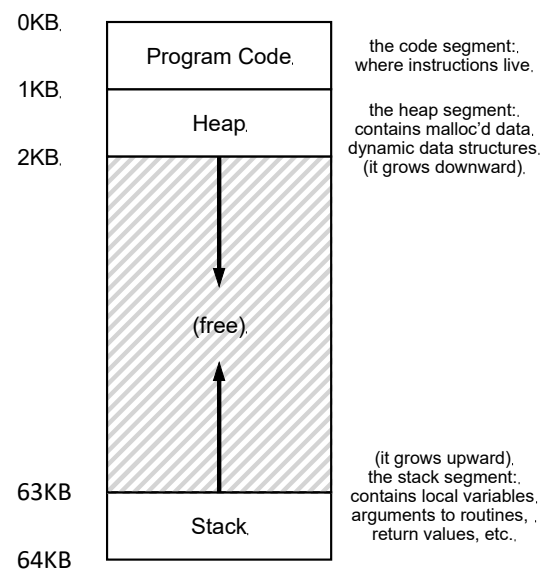
- RAM is cheap, but not as cheap as solid state or cloud storage
- What happens when you run out of RAM?



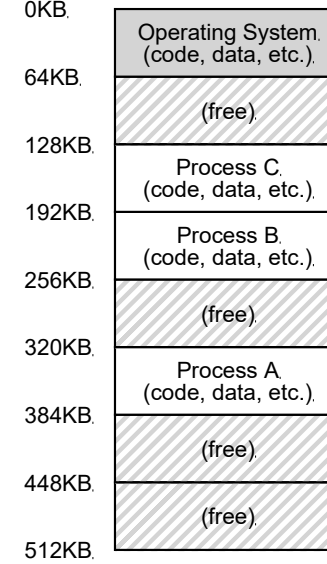
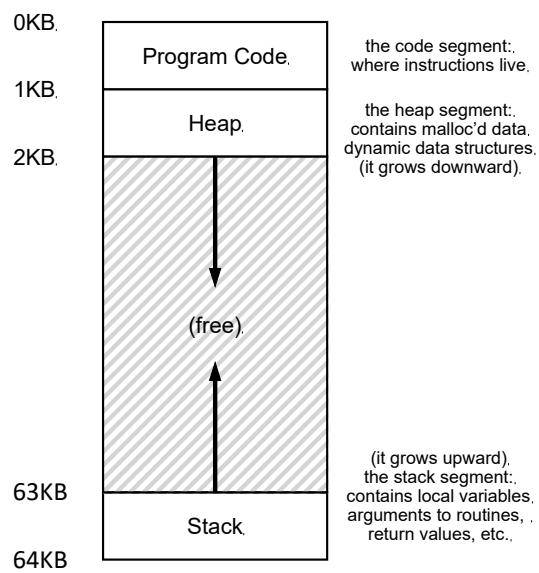
Address Space

- The **abstraction** that the OS is providing to the running program
 - Running program's view of physical memory: contain all the memory state of running program
 - The process is NOT in memory at physical addresses 0 through 64KB
 - **Virtual address** vs. physical address





Q: For the above address space and physical memory, how many bits are there in the virtual address and the physical address respectively?



Q: For the above address space and physical memory, how many bits are there in the virtual address and the physical address respectively?

A: Virtual address has 16 bits; physical address has 19 bits;

Address Space

- Virtualizing memory: the OS builds the abstraction of a private, potentially large address space for multiple running processes (all sharing memory) on top of a single, physical memory.
 - The running program has an illusion that it is loaded into memory at a particular address (e.g., 0x0), and has a potentially very large address space (e.g., 32-bits or 64-bits)
 - What is the size of the address space if a system has 32 bits for a (virtual) address?

Goals of virtualizing memory

- **Transparency**
 - virtualized memory is implemented in a way that is invisible to the running program
 - program behaves as if it has its own private physical memory
 - OS and hardware do all the work to multiplex memory among many different processes, and hence implement the illusion
- **Efficiency** – time and space
- **Protection**
 - Isolation is a key principle in building reliable systems.
 - Two isolated entities cannot affect each other on their failures.
 - OSes strive to isolate processes from each other to prevent one from harming the other.
 - By using memory isolation, the OS further ensures that running programs cannot affect the operation of the underlying OS.

Every address you see is virtual

- **Every address** in a running program is virtual.
 - OS translates the virtual address to physical address

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){

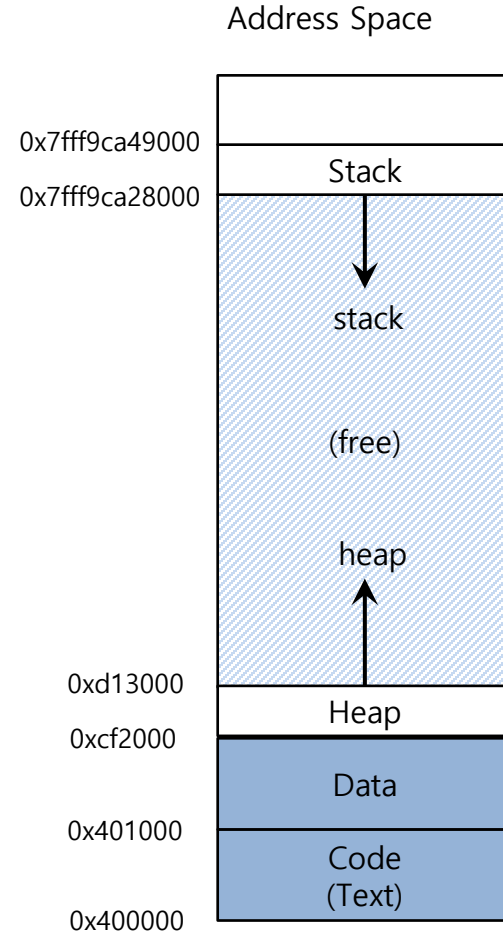
    printf("location of code   : %p\n", (void *) main);
    printf("location of heap   : %p\n", (void *) malloc(1));
    int x = 3;
    printf("location of stack : %p\n", (void *) &x);

    return x;
}
```

A simple program that prints out addresses

Every address you see is virtual

```
location of code   : 0x40057d  
location of heap   : 0xcf2010  
location of stack  : 0x7fff9ca45fcc
```



Memory APIs: memory types

- In running a C program, there are two types of memory that are allocated ...

- Stack (or automatic) memory

- managed **implicitly** by compiler for programmer

```
void func() {  
    int x; // declares an integer on the stack  
    ...  
}
```

- Heap memory

- **explicitly** managed by programmer

```
void func() {  
    int *x = (int *) malloc(sizeof(int));  
    ...  
}
```

Memory APIs: `malloc()`

- Allocate memory on the heap.
 - succeeds and gives you back a pointer to the newly-allocated space
 - fails and returns `NULL`

```
double *d = (double *) malloc(sizeof(double));
```

- Size of memory allocation
 - `sizeof()` operator to request the right amount of space
 - `sizeof()` is a compile-time operator (meaning that the actual size is known at compile time and thus a number is substituted as the argument to `malloc()`).

Memory APIs: `malloc()`

```
1 int *x = malloc(10 * sizeof(int));  
2 printf("%d\n", sizeof(x));  
3 int y[10];  
4 printf("%d\n", sizeof(y));
```

- `sizeof()` operator returns value that can be determined at compile-time

Memory APIs: `free()`

- To free heap memory that is no longer in use, programmers simply call `free()`:

```
int *x = malloc(10 * sizeof(int));
```

```
...
```

```
free(x);
```

- The routine takes one argument, a pointer that was returned by `malloc()`.
 - With `free()`, the size of the allocated region is not passed in, by the user → sizes of each allocated chunk must be tracked by the memory-allocation library itself.

Memory APIs: common errors

- Error 1

```
char *src = "hello";  
char *dst;           // oops! unallocated  
strcpy(dst, src); // segfault and die
```

– **Forgetting to allocate memory**

- Error 2

```
char *src = "hello";  
char *dst = (char *) malloc(strlen(src)); // too small!  
strcpy(dst, src); // work properly, but buffer overflow
```

– **Not allocating enough memory**

Memory APIs: common errors

- Error 3: forgetting to initialize allocated memory
 - Allocated memory contains unknown values that may bring unexpected outcome.
- Error 4: forgetting to free memory
 - Causing memory leak
 - Not all memory has to be explicitly freed (e.g., memory of a short-lived program). But explicitly doing it is a good habit.
- Error 5: freeing memory before you're done with it
 - Dangling pointer
 - Could crash the program, or overwrite valid memory
- Error 6: freeing memory repeatedly
 - Double freeing
 - Unexpected and weird outcomes
- Error 7: calling `free()` incorrectly
 - Invalid freeing: free something that is not returned by `malloc()`